

INF781 — SEGURIDAD DE SOFTWARE

EXAMEN SEGUNDO PARCIAL

Control de Acceso (RBAC) con spatie/laravel-permission

Carrera	Ingeniería Informática — Facultad de Ciencias Puras
Universidad	Universidad Autónoma Tomás Frías (UATF) — Potosí, Bolivia
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Stack	Laravel 13 · PHP 8.3 · spatie/laravel-permission ^7.0
Duración	120 minutos
Puntaje total	100 puntos
Modalidad	Práctica individual en computadora

Nombre del estudiante: _____

Registro / C.I.: _____ Fecha: _____

Contexto del problema

La empresa AlmaTrack S.R.L. administra varios almacenes y necesita un módulo web de control de inventario con acceso restringido por roles. Existen dos universos de usuarios distintos que NO comparten credenciales:

- Personal operativo (almacenistas, supervisores, administradores), que ingresa por la interfaz web tradicional con sesión y cookies.
- Sistemas externos / app móvil de los repartidores, que consumen una API y se autentican con tokens.

Tu tarea es implementar el sistema de autorización completo usando spatie/laravel-permission sobre Laravel 13, contemplando MÚLTIPLES GUARDS, gestión dinámica de roles (CRUD) y protección tanto de rutas web como de endpoints de API.

Recursos permitidos

Documentación oficial de Laravel 13 y de spatie/laravel-permission.

Proyecto base con autenticación con Breeze (guard web) y Sanctum (guard api) configurados.

NO está permitido copiar repositorios completos.

Modelo de roles y permisos requerido

Debes definir los siguientes roles y permisos. Presta atención a que algunos roles pertenecen al guard web y otros al guard api.

Roles por guard

Rol	Guard	Descripción
admin	web	Acceso total: gestiona usuarios, roles y todo el inventario.
supervisor	web	Aprueba movimientos y ve reportes; no gestiona roles.
almacenista	web	Registra entradas/salidas de productos de su almacén.
repartidor	api	Consulta productos y confirma entregas vía token.

Permisos mínimos

Permiso	admin	supervisor	almacenista	repartidor
ver productos	✓	✓	✓	✓ (api)
crear productos	✓	—	—	—
editar productos	✓	✓	—	—
eliminar productos	✓	—	—	—
registrar movimiento	✓	✓	✓	—
aprobar movimiento	✓	✓	—	—

Permiso	admin	supervisor	almacenista	repartidor
confirmar entrega	—	—	—	✓ (api)
gestionar roles	✓	—	—	—

Atención a los guards

Los permisos del guard web y del guard api son independientes en spatie/laravel-permission: aunque un permiso se llame igual, se registra por separado para cada guard. Justifica tus decisiones en los comentarios del código.

Requisitos de implementación

Parte A — Instalación y configuración (15 pts)

1. Instala y configura spatie/laravel-permission ^7.0, publica y ejecuta la migración, y habilita el trait HasRoles en el modelo User. (5 pts)
2. Registra los alias de middleware role, permission y role_or_permission en bootstrap/app.php conforme a Laravel 11+. (5 pts)
3. Crea un seeder que genere todos los roles y permisos de las tablas anteriores, respetando el guard correspondiente de cada uno. (5 pts)

Pista de Laravel 13

Recuerda que el middleware ya NO se registra en app/Http/Kernel.php. Debe ir dentro de `->withMiddleware()` en bootstrap/app.php usando `$middleware->alias([...])`.

Parte B — Protección de rutas web con atributos #[Middleware] (25 pts)

Implementa ProductController (resource) protegido con los atributos #[Middleware] de Laravel 13 — NO uses `$this->middleware()` en el constructor.

4. index/show accesibles por cualquier rol web con permiso 'ver productos'. (6 pts)
5. store/destroy restringidos a 'crear productos' y 'eliminar productos' respectivamente. (7 pts)
6. update restringido a 'editar productos'. (4 pts)
7. Usa el atributo a nivel de método (no solo de clase) y demuestra al menos un caso con role_or_permission. (8 pts)

Parte C — Policy y directivas Blade (20 pts)

8. Crea una MovementPolicy donde 'aprobar movimiento' solo lo pueda ejecutar quien tenga ese permiso, y un almacenista solo pueda registrar movimientos de SU propio almacén (campo warehouse_id). (10 pts)
9. Registra y usa la policy mediante la función can nativa (Gate), no reimplementando la lógica a mano. (4 pts)

10. En la vista Blade, oculta los botones 'Crear', 'Eliminar' y 'Aprobar' usando las directivas @can / @role según corresponda. (6 pts)

Parte D — API con guard api (20 pts)

11. Expón GET /api/products protegido para el guard api con permiso 'ver productos'. (6 pts)
12. Expón POST /api/deliveries/{id}/confirm que exige el permiso 'confirmar entrega' del guard api. (8 pts)
13. Asegura que un token de repartidor NO pueda crear ni eliminar productos, devolviendo 403. Explica por qué la separación de guards lo garantiza. (6 pts)

Parte E — CRUD dinámico de roles (15 pts)

14. Implementa un RoleController (solo para 'gestionar roles') que permita crear, listar, editar y eliminar roles del guard web, y asignar/quitar permisos a un rol. (9 pts)
15. Tras asignar permisos, refresca la caché de permisos de Spatie para que los cambios apliquen sin reiniciar. (3 pts)
16. Impide eliminar el rol 'admin' (rol crítico del sistema). (3 pts)

Parte F — Pregunta teórica (5 pts)

Responde brevemente en un archivo RESPUESTAS.md: ¿Por qué spatie/laravel-permission cachea los permisos y qué problema de seguridad o de consistencia podría aparecer si olvidas limpiar esa caché tras un cambio de permisos en producción? (máx. 8 líneas)

Entregables

- Repositorio Git con el proyecto Laravel 13 funcional.
- Migración + seeder ejecutables con php artisan migrate --seed.
- Colección de pruebas (Postman/Bruno o tests de PHPUnit/Pest) que demuestre los 403 esperados.
- Archivo RESPUESTAS.md con la pregunta teórica de la Parte F.

Criterio de seguridad transversal (penalización)

Se descontarán hasta 10 puntos si la lógica de autorización se evade fácilmente: rutas sin proteger, validación solo en el frontend, o checks de rol hechos con comparaciones de strings (ej. `if($user->role == 'admin')`) en lugar de usar el sistema de permisos.

Rúbrica de evaluación

Parte	Criterio	Puntos
A	Instalación, middleware y seeder por guard	15

Parte	Criterio	Puntos
B	Rutas web con atributos #[Middleware]	25
C	Policy de almacén + directivas Blade	20
D	API con guard api y aislamiento de tokens	20
E	CRUD dinámico de roles + caché + rol protegido	15
F	Pregunta teórica (caché de permisos)	5
	TOTAL	100

Bandas de calificación

90–100: Dominio sólido. Guards bien separados, atributos y políticas correctos.

70–89: Funcional con detalles menores (caché olvidada, un guard mal asignado).

50–69: Implementación parcial; faltan API o CRUD de roles.

< 50: Autorización ausente o evadible; no cumple el objetivo de seguridad.